

Session 6: Encapsulation & Abstraction Topics: Access modifiers: public, private, protected. Abstract classes & pure virtual functions (conceptual). Example: Class Platform with abstract upload() method.

Tasks

- 1) Create a class called Song in your preferred OOP language with private properties title and artist. Add public getter and setter methods to access and modify these properties, then create an object and update its title.

Code-

```
#include<iostream>
#include<string>

using namespace std;

class song{

private:
string title;
string artist;

public:
void setttitle(string t){
    title =t;

}
string gettitle(){
    return title;
}
void setartist(string a){
    artist=a;
}
string getartist(){
```

```

        return artist;
    }
};

int main(){

    song s1;
    s1.settitle("shape of you");
    s1.setartist("Ed Sheeran");

    cout<<"\n original song"<<endl;
    cout<<"title   :"<<s1.gettitle()<<endl;
    cout<<"artist  :"<<s1.getartist()<<endl;

    s1.settitle("peffect");

    cout<<"\n after updating artist"<<endl;
    cout<<"ttitle  :"<<s1.gettitle()<<endl;
    cout<<"artist  :"<<s1.getartist()<<endl;

return 0;
}

```

Output:

```

PS C:\Users\Admin\OneDrive\Desktop\cpp> g++ demo31.cpp -o demo31 -static
>> .\demo31

    original song
    title       :shape of you
    artist      :Ed Sheeran

    after updating artist
    ttitle      :peffect
    artist      :Ed Sheeran
PS C:\Users\Admin\OneDrive\Desktop\cpp> 

```

- 2) Build a class called InstaStory with a protected property storyViews. Create a subclass called SponsoredStory that can access and display the storyViews value.**

Code-

```
#include<iostream>

#include<string>

using namespace std;
```

```
/* .
```

Build a class called
InstaStory with a protected property storyViews. Create a subclass
called SponsoredStory that can access and display the storyViews value.*/

```
class InstaStory{
    protected:
        int storyViews;

    public:
        InstaStory(int s)
        {
            storyViews=s;
        }
}
```

```
};

class SponsoredStory:public InstaStory{

public:

    SponsoredStory(int s):InstaStory(s){

    }

    void dispaly(){

        cout<<"storyviews vlaue :"<<storyViews<<endl;

    }

};

int main(){

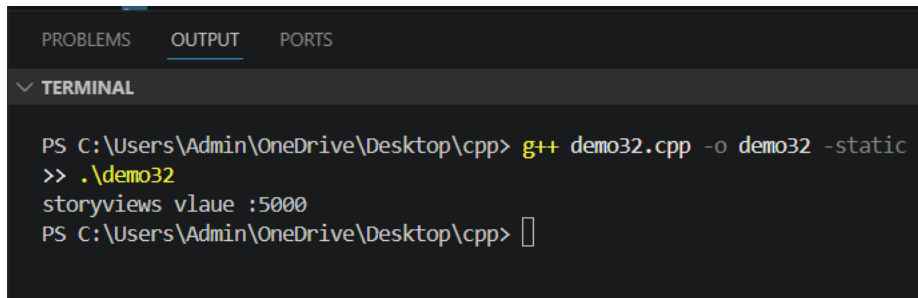
    SponsoredStory s1(5000);

    s1.dispaly();

    return 0;

}
```

Output:



```
PROBLEMS  OUTPUT  PORTS
▼ TERMINAL

PS C:\Users\Admin\OneDrive\Desktop\cpp> g++ demo32.cpp -o demo32 -static
>> .\demo32
storyviews vlaue :5000
PS C:\Users\Admin\OneDrive\Desktop\cpp> 
```

3) Imagine a Flipkart-like app: create an abstract class Product with an abstract method upload(). Then, create two subclasses, Electronics and Clothing, that each implement the upload() method to print a different upload message

Code-

```
#include<iostream>
#include<string>
using namespace std;
```

```
class UserProfile {
private:
```

```
    string number;
```

```
public:
```

```
    void setnumber(string n){
        number=n;
```

```
    }
```

```
    string getnumber(){
        return number;
```

```
    }
```

```
};
```

```
int main(){
```

```
    UserProfile u1;
```

```
    u1.setnumber("8128005152");
```

```
    cout<<"phone number :"<<u1.getnumber()<<endl;
```

```
    return 0;
```

}

Output:

```
PROBLEMS  OUTPUT  PORTS
✓ TERMINAL

PS C:\Users\Admin\OneDrive\Desktop\cpp> g++ demo34.cpp -o demo34 -static
>> .\demo34
phone number :8128005152
PS C:\Users\Admin\OneDrive\Desktop\cpp> |
```

TOPS

5) Explain in your own words the difference between encapsulation and abstraction, using examples from any social media app you use daily (like Instagram or WhatsApp).

Encapsulation

Protects and controls access to data.

Focuses on **how data is accessed or modified**.

Achieved using private, protected, and public.

Abstraction

Hides unnecessary implementation details.

Focuses on **what the user can do** without knowing how it works.

Achieved using abstract classes, interfaces, and simplified methods.

1) Encapsulation-

```
class UserProfile {  
private:  
    string phoneNumber;  
  
public:  
    void setPhoneNumber(string number) {  
        phoneNumber = number;  
    }  
  
    string getPhoneNumber() {  
        return phoneNumber;  
    }  
};
```

Explanation-

Suppose Instagram has a UserProfile class containing a user's phone number. The phone number can be made **private**, so other parts of the program cannot directly change it. Instead, Instagram provides methods such as setPhoneNumber() and getPhoneNumber() to safely update or retrieve it.

2. Abstraction:

When I upload a photo on Instagram, I simply click **Post**. I don't need to know how Instagram compresses the image, stores it on servers, processes it, and delivers it to followers.

Instagram gives me a simple action—`post()`—while hiding the complicated internal process.

Easy way to remember

- **Encapsulation = Protect the data** 🔒
- **Abstraction = Hide the complexity** 🎯